

Exception Handling

Write Robust, Production-Ready Python

■ LEARNING OUTCOME

By the end of this module you will handle all Python exception types, create custom exceptions, use try/except/else/finally correctly, apply context managers, use logging professionally, and write defensive code.

■ Skills You Will Gain

- Use try/except/else/finally correctly
- Handle specific exception types
- Create custom exception classes
- Use context managers with 'with' statement
- Write your own context managers
- Set up professional logging

■ Exceptions are not failures -- they are signals. Well-handled exceptions make programs robust and debuggable. In production Python, exception handling and logging are as important as the logic itself.

SECTION 1

Exception Hierarchy and try/except

Exception	When Raised
Exception	Base for all non-system-exiting exceptions
ValueError	Right type but wrong value: <code>int('abc')</code> , <code>sqrt(-1)</code>
TypeError	Wrong type for operation: <code>1 + 'a'</code> , <code>len(42)</code>
KeyError	Dict key not found: <code>d['missing']</code>
IndexError	List index out of range: <code>lst[100]</code>
AttributeError	Object has no attribute: <code>None.upper()</code>
FileNotFoundError	File does not exist
PermissionError	No permission to read/write file
ZeroDivisionError	Division by zero: <code>10/0</code>
OverflowError	Result too large for numeric type
ImportError	Module not found or import failed
StopIteration	Iterator exhausted -- <code>next()</code> called on empty
RuntimeError	Generic runtime error
NotImplementedError	Abstract method not implemented
OSError/IOError	OS-level error -- file system, network

```
# try / except / else / finally
try: number = int(input("Enter a number: "))
result = 100 / number
except ValueError: print("Invalid input -- not a number")
except ZeroDivisionError: print("Cannot divide by zero")
except (TypeError, AttributeError) as e: print(f"Type error: {e}")
except Exception as e: print(f"Unexpected error: {type(e).__name__}: {e}")
raise # re-raise -- let caller handle it
else: print(f"Result: {result}")
# runs ONLY if no exception
finally: print("Always runs -- cleanup code here")
# Catching exception info
try: x = int("abc")
except ValueError as e:
    print(e.args) # ('invalid literal...',)
    print(str(e)) # invalid literal for int()...
    print(type(e).__name__) # ValueError
import traceback
traceback.print_exc() # full stack trace to stderr
# RAISE -- throw an exception
def divide(a, b):
    if not isinstance(b, (int, float)):
        raise TypeError(f"b must be numeric, got {type(b).__name__}")
    if b == 0:
        raise ZeroDivisionError("b cannot be zero")
    return a / b
# raise from -- chained exceptions
try: data = load_file("data.json")
except FileNotFoundError as e:
    raise RuntimeError("Config file missing") from e
# Shows:
RuntimeError: Config file missing
# During handling of: FileNotFoundError: ...
```

SECTION 2

Custom Exceptions and Context Managers

```
# CUSTOM EXCEPTIONS
class AppError(Exception):
    """Base exception for this application."""
    pass
class ValidationError(AppError):
    def __init__(self, field: str, message: str):
        super().__init__(f"Validation failed for '{field}': {message}")
        self.field = field
        self.message = message
class NotFoundError(AppError):
    def __init__(self, resource: str, identifier):
```

```

super().__init__(f"{resource} with id '{identifier}' not found") self.resource = resource
self.identifier = identifier class APIError(AppError): def __init__(self, status_code: int,
message: str): super().__init__(f"API error {status_code}: {message}") self.status_code =
status_code # Using custom exceptions try: user = get_user(user_id) except NotFoundError as e:
print(f"User not found: {e.identifier}") except ValidationError as e: print(f"Bad input for field:
{e.field}") except AppError as e: print(f"App error: {e}") # CONTEXT MANAGERS -- with statement #
Using contextlib.contextmanager decorator from contextlib import contextmanager, suppress import
time @contextmanager def timer(label=""): """Context manager to time a code block.""" start =
time.perf_counter() try: yield # code inside "with" runs here finally: elapsed =
time.perf_counter() - start print(f"{label}: {elapsed:.4f}s") with timer("Data processing"): result
= [x**2 for x in range(1_000_000)] # suppress -- ignore specific exceptions with
suppress(FileNotFoundError): os.remove("temp_file.txt") # no error if file doesn't exist # Custom
context manager class class ManagedResource: def __enter__(self): print("Acquiring resource")
return self # value bound to "as" variable def __exit__(self, exc_type, exc_val, exc_tb):
print("Releasing resource") if exc_type is ValueError: print("Handling ValueError") return True #
suppress the exception return False # re-raise other exceptions with ManagedResource() as r: pass

```

SECTION 3

Logging -- Professional Error Tracking

```

import logging import logging.handlers from pathlib import Path # LOGGING LEVELS: DEBUG < INFO <
WARNING < ERROR < CRITICAL # Basic configuration logging.basicConfig( level=logging.INFO,
format="%(asctime)s %(name)s %(levelname)s %(message)s", datefmt="%Y-%m-%d %H:%M:%S", ) #
Module-level logger -- best practice logger = logging.getLogger(__name__) # Use logger, not print()
logger.debug("Processing item %s", item_id) logger.info("User %s logged in", username)
logger.warning("Disk usage at %d%%", disk_pct) logger.error("Failed to connect: %s", str(e))
logger.critical("Database unreachable!") # Exception logging try: risky_operation() except
Exception: logger.exception("Unexpected error in risky_operation") # includes traceback # File +
console logging setup def setup_logging(log_file="app.log", level=logging.INFO): fmt =
logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s") root = logging.getLogger()
root.setLevel(level) # Console handler ch = logging.StreamHandler() ch.setFormatter(fmt)
root.addHandler(ch) # Rotating file handler -- max 10MB, keep 5 backups fh =
logging.handlers.RotatingFileHandler( log_file, maxBytes=10*1024*1024, backupCount=5 )
fh.setFormatter(fmt) root.addHandler(fh) setup_logging()

```

■ PRACTICE Q & A

Q1. What is the difference between `except Exception` and bare `except`?

A: `except Exception`: catches all non-system-exiting exceptions. Bare `'except:'` catches EVERYTHING including `KeyboardInterrupt` and `SystemExit` -- this is almost never what you want. Always use `'except Exception'` at minimum.

Q2. What does the `else` clause do in `try/except`?

A: The `else` block runs only if NO exception was raised in the `try` block. Use it for code that should only run on success -- separates 'try this' from 'do this on success'.

Q3. When should you create a custom exception class?

A: When you need to: carry extra data (field name, status code), create a hierarchy of errors for your library, allow callers to catch your specific errors without catching all exceptions, or when the built-in exceptions don't describe the error clearly.

Q4. What is a context manager and why is it better than `try/finally`?

A: A context manager (with `statement`) guarantees `__exit__` is called even if an exception occurs -- equivalent to `try/finally` but cleaner. Use for: file handles, database connections, locks, timers, any resource that needs cleanup.

Q5. What is the difference between logging and `print` for debugging?

A: `print()` has no severity level, no timestamp, no module info, and no easy way to disable. logging has levels (DEBUG/INFO/WARNING), timestamps, module names, file output, log rotation, and can be completely disabled in production without changing code.

<code>try: / except ValueError: / else: / finally:</code>	Full exception block
<code>except (TypeError, ValueError) as e:</code>	Catch multiple types
<code>raise ValueError('message')</code>	Raise exception
<code>raise RuntimeError('msg') from e</code>	Chained exception
<code>except Exception: raise</code>	Re-raise caught exception
<code>class AppError(Exception):</code>	Custom exception
<code>@contextmanager</code>	Create context manager from fn
<code>with suppress(FileNotFoundError):</code>	Ignore specific exception
<code>logger = logging.getLogger(__name__)</code>	Module-level logger
<code>logger.exception('msg')</code>	Log with full traceback
<code>logging.basicConfig(level=logging.INFO)</code>	Basic logging setup
<code>RotatingFileHandler</code>	Auto-rotate log files